

PRD — Fintech Ledger / Wallet API

Project codename: LedgerCore (rename freely) **Stack:** NestJS · Prisma · PostgreSQL · JWT · class-validator · BullMQ · Docker **Status:** Draft v1 — companion to the scope document
Author: Bolu · First NestJS backend project (learning-through-building)

1. Purpose & Summary

A backend API for a digital wallet system. Users register, open wallet accounts, deposit money (via a simulated payment provider), transfer to other users, and withdraw. Every movement of money is recorded as **double-entry ledger entries** — the source of truth for balances is the ledger, not a mutable balance column.

The primary success criterion is **correctness under failure and concurrency**: no negative balances, no double-spends, no double-applied retries, no half-completed transfers.

What "done" means

The project is complete when all acceptance criteria in §10 pass, including automated tests that fire concurrent transfers at the same account and prove the locking strategy holds.

2. Users & Roles

Role	Description	Capabilities
User	Wallet owner	Register, log in, create/view own accounts, deposit, withdraw, transfer, view own transactions & statements
Admin	Operator	View any account, freeze/unfreeze accounts, view audit logs, trigger reconciliation
System	Internal actor	Owns system accounts (see §5.3), processes webhooks and background jobs

There is no customer support role, no KYC tiers, no merchant role — deliberately out of scope.

3. Core Concepts (read before the schema)

3.1 Money representation

- All amounts are stored as **integers in minor units** (kobo). ₦1,500.00 = `150000`.
- Never use floats for money. Prisma type: `BigInt` (or `Int` if you cap max amounts).
- Single currency for v1: `NGN`. The `currency` field exists on accounts so multi-currency is possible later, but **cross-currency transfers are rejected** in v1.

3.2 Double-entry ledger

- Every transaction produces **two or more ledger entries** (debits and credits).
- Convention for this project: **DEBIT = money leaves the account, CREDIT = money enters the account**. Store amounts as positive integers plus a `direction` enum.
- **Invariant:** for every transaction, $\text{sum(credits)} - \text{sum(debits)} = 0$.
- Ledger entries are **append-only and immutable**. No `UPDATE` or `DELETE` on the ledger table, ever. Mistakes are corrected by a new reversing transaction.

3.3 Balances

- **Source of truth:** $\text{balance} = \text{SUM(credits)} - \text{SUM(debits)}$ over an account's ledger entries.
- **Hybrid model (recommended):** each account also carries a `cachedBalance` column updated inside the same DB transaction as the ledger entries. A reconciliation job (§8, Phase 6/7) verifies `cachedBalance` matches the ledger sum and flags drift.
- **Invariant:** available balance never goes below zero (no overdrafts in v1).

3.4 System accounts

Deposits and withdrawals involve money entering/leaving the platform, so double entry needs a counterpart account. Seed these at startup:

Account	Purpose
<code>SYSTEM_CASH</code>	Counterpart for deposits (provider settles money in) and withdrawals (money out)
<code>SYSTEM_FEES</code>	Collects fees, if you add them later (optional in v1)

Example — user deposits ₦5,000:

```
DEBIT  SYSTEM_CASH      500000
CREDIT user_account     500000
```

Example — user A transfers ₦2,000 to user B:

```
DEBIT  account_A        200000
CREDIT account_B        200000
```

4. Functional Requirements

Written as user stories with acceptance criteria. IDs are referenced in §10.

4.1 Auth & Accounts

FR-1: Registration. As a visitor, I can register with email + password.

- Password hashed with bcrypt/argon2. Duplicate email → (409).
- On registration, one default NGN wallet account is auto-created for the user.

FR-2: Login. As a user, I can log in and receive a JWT access token.

- Invalid credentials → (401) with a generic message (no "email exists" leakage).
- Token contains (sub) (userId) and (role). Expiry: 15 min access token. (Refresh tokens: optional stretch goal.)

FR-3: Account ownership. As a user, I can only read or transact on my own accounts.

- Accessing another user's account → (403) (or (404) to avoid enumeration — pick one and be consistent; this PRD says (404)).

FR-4: Account lifecycle. Accounts have status (ACTIVE | FROZEN | CLOSED).

- Only admins can freeze/unfreeze. Frozen accounts reject all balance-changing operations with (AccountFrozenException) → (403).

4.2 Transfers

FR-5: Transfer. As a user, I can transfer from my account to another user's account by account number.

- Request: (sourceAccountId), (destinationAccountNumber), (amount), (narration?), header (Idempotency-Key) (required).
- Rules: amount > 0; source ≠ destination; both accounts (ACTIVE); same currency; source balance ≥ amount.
- On success: one (Transaction) row (type (TRANSFER), status (COMPLETED)) + two ledger entries + both cached balances updated — **all in one DB transaction**.
- Insufficient funds → (422 InsufficientFundsException). Destination not found → (404).

FR-6: Atomicity. If anything fails mid-transfer (crash, constraint violation), **nothing** is applied. No orphaned ledger entries, no half-updated balances.

FR-7: Concurrency safety. Two simultaneous transfers from the same account cannot overdraw it.

- Strategy: (SELECT ... FOR UPDATE) on the source account row (via Prisma (\$queryRaw) inside (\$transaction)), locking accounts in a **consistent order (by id)** to avoid deadlocks. Optimistic locking with a (version) column is an acceptable alternative — pick one and document why in the README.

FR-8: Idempotency. Retrying a transfer with the same (Idempotency-Key) returns the **original response** and applies the transfer **once**.

- Same key + different request body → (409 IdempotencyConflict).
- Keys are scoped per user, expire after 24h.
- A request that arrives while the original is still processing → (409) with a "retry later" message (simplest correct behavior).

4.3 Deposits (webhook-driven)

FR-9: Initiate deposit. As a user, I can initiate a deposit and receive a payment (`reference`) (simulating a Paystack/Flutterwave checkout).

- Creates a `Transaction` with type `DEPOSIT`, status `PENDING`. No ledger entries yet.

FR-10: Deposit confirmation webhook. A simulated provider POSTs to `/webhooks/payments` to confirm payment.

- Signature verified via HMAC-SHA512 of the raw body with a shared secret (Paystack-style `x-paystack-signature` pattern). Invalid signature → `401`, body not processed.
- Valid event: transaction moves `PENDING → COMPLETED`, ledger entries written (`DEBIT SYSTEM_CASH / CREDIT user account`), cached balance updated — atomically.
- Webhook idempotency:** the same provider event delivered twice is processed once (dedupe on provider `eventId`).
- Unknown reference → respond `200` (acknowledge) but log and store the event for investigation. Providers retry on non-2xx; you don't want infinite retries on a permanently-bad event.

4.4 Withdrawals

FR-11: Withdrawal. As a user, I can withdraw to a (simulated) bank account.

- Funds move immediately: `DEBIT user account / CREDIT SYSTEM_CASH`, transaction status `COMPLETED`. (A `PENDING → settlement webhook` flow is a stretch goal; immediate settlement keeps v1 finishable.)
- Same idempotency and concurrency rules as transfers.

4.5 History & Statements

FR-12: Transaction history. As a user, I can list my account's transactions, paginated, filterable by type, status, and date range. Cursor or offset pagination — offset is fine for v1.

FR-13: Monthly statement. As a user, I can request a statement for a month; a background job (BullMQ) generates it (JSON or CSV) and I can fetch the result.

- Statement includes opening balance, closing balance, and all entries — derived from the ledger, so opening + entries must equal closing (assert this in the job).

4.6 Admin & Audit

FR-14: Audit log. Every balance-changing action and every admin action writes an immutable audit record: actor, action, entity, payload snapshot, timestamp.

- Written in the same DB transaction as the action where feasible.

FR-15: Admin endpoints. Admins can list/search accounts, view any account's ledger, freeze/unfreeze, read audit logs, and trigger reconciliation.

FR-16: Reconciliation. An admin-triggered (and nightly scheduled) job recomputes each account's ledger sum and compares to `cachedBalance`. Drift → flagged in a

`reconciliation_reports` record + logged. It does **not** auto-fix.

5. Data Model (Prisma-oriented)

5.1 Tables

users

column	type	notes
id	uuid pk	
email	text unique	lowercase before store
passwordHash	text	
role	enum (USER ADMIN)	default USER
createdAt / updatedAt	timestamp tz	

accounts

column	type	notes
id	uuid pk	
userId	uuid fk → users, nullable	null for system accounts
accountNumber	text unique	10-digit, generated
type	enum (USER SYSTEM)	
currency	char(3)	(NGN)
status	enum (ACTIVE FROZEN CLOSED)	default ACTIVE
cachedBalance	bigint	minor units; default 0
version	int	only if you choose optimistic locking
createdAt / updatedAt	timestamp tz	

transactions

column	type	notes
id	uuid pk	
type	enum (TRANSFER DEPOSIT WITHDRAWAL REVERSAL)	
status	enum (PENDING COMPLETED FAILED REVERSED)	
reference	text unique	human-readable, e.g. (TXN-20260804-XXXX)
amount	bigint	minor units
currency	char(3)	
narration	text nullable	
initiatedByUserId	uuid fk nullable	null for webhook/system-initiated
metadata	jsonb	provider refs etc.
createdAt / updatedAt	timestamptz	

ledger_entries (append-only)

column	type	notes
id	uuid pk	
transactionId	uuid fk → transactions	
accountId	uuid fk → accounts	
direction	enum (DEBIT CREDIT)	
amount	bigint	always positive
balanceAfter	bigint	account's cached balance after this entry — great for statements & debugging
createdAt	timestamptz	no updatedAt: immutable

idempotency_keys

column	type	notes
key	text	pk composite with userId
userId	uuid	
requestHash	text	sha256 of method+path+body
status	enum	PROCESSING COMPLETED
responseStatus	int nullable	
responseBody	jsonb nullable	
expiresAt	timestamptz	createdAt + 24h

webhook_events

column	type	notes
id	uuid pk	
provider	text	mockpay
providerEventId	text unique	dedupe key
payload	jsonb	raw
signatureValid	bool	
processedAt	timestamptz nullable	

audit_logs (append-only)

column	type	notes
id	uuid pk	
actorType	enum (USER ADMIN SYSTEM)	
actorId	uuid nullable	
action	text	e.g. (TRANSFER_COMPLETED), (ACCOUNT_FROZEN)
entityType / entityId	text / uuid	
payload	jsonb	snapshot
createdAt	timestamp tz	

reconciliation_reports

column	type	notes
id	uuid pk	
runAt	timestamp tz	
accountsChecked	int	
driftCount	int	
details	jsonb	per-account drift info

5.2 Database-level guards (defense in depth)

- (CHECK (amount > 0)) on ledger_entries and transactions.
- (CHECK (cachedBalance >= 0)) on user accounts (system accounts may go negative — SYSTEM_CASH naturally will).
- Unique constraint on (userId, key) in idempotency_keys and on (providerEventId) in webhook_events — the DB, not just app code, prevents duplicates.
- No FK cascade deletes anywhere near ledger/transactions.

5.3 Seed data

- 1 admin user, 2 test users (each with a default account), (SYSTEM_CASH) account. Seed script: (prisma/seed.ts).

6. API Endpoints

Base path `/api/v1`. All authenticated unless marked public. Errors follow the format in §7.

Auth

Method	Path	Auth	Notes
POST	<code>/auth/register</code>	public	FR-1
POST	<code>/auth/login</code>	public	FR-2, returns <code>{ accessToken }</code>
GET	<code>/auth/me</code>	user	current user + accounts

Accounts

Method	Path	Auth	Notes
GET	<code>/accounts</code>	user	own accounts
GET	<code>/accounts/:id</code>	user (owner)	includes <code>cachedBalance</code>
GET	<code>/accounts/:id/balance</code>	user (owner)	<code>{ cached, ledgerDerived }</code> — showing both is a great debugging/demo feature
GET	<code>/accounts/:id/transactions</code>	user (owner)	FR-12, paginated + filters
GET	<code>/accounts/:id/statements/:period</code>	user (owner)	FR-13, <code>period=2026-08</code> ; 202 + job id if not ready

Money movement

Method	Path	Auth	Notes
POST	<code>/transfers</code>	user	FR-5.8, requires <code>Idempotency-Key</code> header
POST	<code>/deposits</code>	user	FR-9, returns <code>{ reference }</code>
POST	<code>/withdrawals</code>	user	FR-11, requires <code>Idempotency-Key</code>
GET	<code>/transactions/:id</code>	user (participant)	with its ledger entries

Webhooks

Method	Path	Auth	Notes
POST	<code>/webhooks/payments</code>	HMAC signature	FR-10; must read raw body for signature verification (Nest: <code>rawBody: true</code> in <code>NestFactory.create()</code>)

Admin

Method	Path	Auth	Notes
GET	<code>/admin/accounts</code>	admin	search/paginate
GET	<code>/admin/accounts/:id/ledger</code>	admin	full ledger
PATCH	<code>/admin/accounts/:id/status</code>	admin	freeze/unfreeze, FR-4
GET	<code>/admin/audit-logs</code>	admin	filterable
POST	<code>/admin/reconciliation/run</code>	admin	FR-16
GET	<code>/admin/reconciliation/reports</code>	admin	

Swagger (`@nestjs/swagger`) documents everything at `/docs` from Phase 1.

7. Error Handling

Standard error envelope (via a global exception filter):

```
json
{
  "statusCode": 422,
  "error": "INSUFFICIENT_FUNDS",
  "message": "Insufficient funds: balance 150000, attempted 200000",
  "timestamp": "2026-08-04T10:00:00Z",
  "path": "/api/v1/transfers"
}
```

Exception (custom class)	HTTP	Code
ValidationException (class-validator)	400	VALIDATION_ERROR
InvalidCredentials	401	INVALID_CREDENTIALS
InvalidWebhookSignature	401	INVALID_SIGNATURE
AccountFrozenException	403	ACCOUNT_FROZEN
ForbiddenRole	403	FORBIDDEN
AccountNotFound (incl. not-owner)	404	ACCOUNT_NOT_FOUND
DuplicateEmail	409	EMAIL_EXISTS
IdempotencyConflict	409	IDEMPOTENCY_CONFLICT
IdempotencyInProgress	409	REQUEST_IN_PROGRESS
InsufficientFundsException	422	INSUFFICIENT_FUNDS
SelfTransferException	422	SELF_TRANSFER
CurrencyMismatch	422	CURRENCY_MISMATCH
Unhandled	500	INTERNAL_ERROR (never leak stack traces)

Validation rules (DTOs, class-validator): amount is a positive integer (minor units), max single transaction ₦10,000,000 (1000000000), narration ≤ 140 chars, email format, password ≥ 8 chars.

8. Background Jobs (BullMQ)

Job	Queue	Trigger	Behavior
Statement generation	statements	user request (FR-13)	derive from ledger, store result, retry ×3 with backoff
Nightly reconciliation	reconciliation	cron 0 2 * * * + admin manual	FR-16
Idempotency-key cleanup	maintenance	cron hourly	delete expired keys
Scheduled transfers	transfers	stretch goal	user schedules a future-dated transfer

All jobs must be idempotent themselves (a retried statement job must not produce duplicates).

9. Non-Functional Requirements

- **Correctness over performance.** No caching layers, no read replicas. Target is a single Postgres instance behaving correctly.
 - **Config:** all secrets via env (`@nestjs/config`) + validation of env vars at boot — fail fast on missing config).
 - **Security:** helmet, rate limiting on auth + money endpoints (`@nestjs/throttler`), bcrypt/argon2, no secrets in logs, no stack traces in responses.
 - **Logging:** structured logs (pino via `nestjs-pino`) with a request id; every money movement logs transaction reference + amounts.
 - **Docker:** `docker-compose.yml` with app + postgres + redis. One-command startup.
 - **CI:** GitHub Actions — lint, typecheck, test on every push.
 - **Docs:** README with architecture diagram, locking-strategy rationale, and "how to run the concurrency demo" — this doubles as portfolio material.
-

10. Acceptance Criteria & Test Plan

Phase 8 proves everything. Minimum test suite:

Unit (services, mocked Prisma)

- Transfer validation: negative amount, self-transfer, currency mismatch, frozen account.
- Ledger math: entries always net to zero.

Integration (real Postgres via docker, e2e with supertest)

- ☒ Register → login → auto-created account visible.
- ☒ Deposit flow: initiate → webhook with valid signature → balance credited; invalid signature → 401 and no balance change; duplicate (`providerEventId`) → applied once.
- ☒ Transfer happy path: balances move, exactly 2 ledger entries, sum = 0, (`balanceAfter`) correct.
- ☒ Insufficient funds → 422, zero side effects.
- ☒ Idempotency: same key replayed 5× → one transaction, identical responses; same key + different body → 409.
- ☒ Ownership: user B reading/transacting on user A's account → 404/403.
- ☒ Frozen account: transfer in/out rejected.

Concurrency (the money test — literally)

- ☒ Account with ₦1,000. Fire **10 parallel transfers of ₦300** (`Promise.all` against the running app). Exactly 3 succeed, 7 fail with `INSUFFICIENT_FUNDS`, final balance ₦100, ledger sum matches.

- ✓ Two opposite transfers between the same two accounts simultaneously ($A \rightarrow B$ and $B \rightarrow A$) $\rightarrow$ no deadlock (proves consistent lock ordering).
- ✓ Same idempotency key sent concurrently $\times 5 \rightarrow$ one applied.

Reconciliation

- ✓ Manually corrupt a `cachedBalance` in test DB $\rightarrow$ job flags exactly that account.

Definition of done per FR: the FR's criteria pass in CI.

11. Phase Plan (mapped to the scope doc)

Phase	Focus	Key deliverable / exit criterion
1 (4-6d)	Nest scaffold, Prisma schema, users/accounts CRUD, DTOs, Swagger	Schema migrated; register/login-less CRUD works; <code>/docs</code> live
2 (4-6d)	JWT auth, guards, RBAC, ownership	FR-1..4 pass; e2e auth tests green
3 (5-7d)	Ledger service, transfers, atomicity	FR-5..6; ledger invariants enforced; <code>balanceAfter</code> correct
4 (5-8d)	Locking + idempotency	FR-7..8; manual concurrency script demonstrates safety
5 (2-3d)	Deposit init + webhook + signature	FR-9..10; withdrawal (FR-11)
6 (2d)	Audit logging	FR-14; admin endpoints FR-15
7 (3-4d)	BullMQ: statements, reconciliation, cleanup	FR-13, FR-16
8 (5-7d)	Full test suite incl. concurrency tests in CI	§10 green in GitHub Actions
9 (2-3d)	Docker polish, CI, deploy (Railway/Render), README	One-command run; public URL; portfolio-ready README

Learning checkpoints (since this is your first Nest project): after Phase 1 you should be able to explain modules/providers/DI without notes; after Phase 2, guards vs interceptors vs middleware and where each runs in the request lifecycle; after Phase 4, exactly why your chosen locking strategy prevents double-spend and what its failure mode is. If you can't, pause and consolidate before moving on — these are the interview questions this project exists to answer.

12. Out of Scope (unchanged from scope doc)

Microservices/message brokers, distributed locks (Redlock), Prometheus/Grafana, GraphQL, Kubernetes, multi-currency FX, real payment provider integration, KYC, fees, overdrafts, refunds beyond simple reversal.

13. Open Decisions (resolve during Phase 1, write the answer in the README)

1. **Pessimistic** (`FOR UPDATE`) vs **optimistic** (`version`) **locking** — PRD default: pessimistic; simpler to reason about for a first implementation.
2. `BigInt` vs `Int` for amounts — PRD default: `BigInt` (learn the JSON serialization quirk: `BigInt` doesn't serialize natively; you'll need a transform).
3. **404** vs **403** for non-owned accounts — PRD default: 404.
4. Refresh tokens now or stretch goal — PRD default: stretch goal.